ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BTL

Phát triển Ứng dụng Internet of Things
CO3037

Lớp: A01

GVHD: Lê Trọng Nhân

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Thanh Toàn	2313492	TN01	Kỹ thuật máy tính
2	Lê Công Vinh	2313912	TN01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Mục lục

Danh sách hình vẽ **3**
1 Giới thiệu chung **4**
1.1 Mục tiêu dự án 4
1.2 Phạm vi thực hiện 4
2 Thiết kế hệ thống **5**
2.1 Kiến trúc hệ thống 5
2.2 Sơ đồ khối và luồng dữ liệu 5
2.3 Danh sách linh kiện phần cứng 6
3 Triển khai các Task **7**
3.1 Task 1,2, 3: Thu thập dữ liệu và trực quan hóa 7
3.1.1 Tiến trình Thu thập Dữ liệu 7
3.1.2 Hiển thị trên LED đơn với các trạng thái nhiệt độ khác nhau 7
3.1.3 Hiển thị RGB LED tương ứng với các trạng thái độ ẩm khác nhau . 9
3.1.4 Hiển thị LCD 10
3.2 Task 4: Web Server ở chế độ Access Point 12
3.2.1 Mục tiêu và chế độ vận hành 12
3.2.2 Giao diện chính của Web Server 12
3.2.3 Trang cấu hình và quy trình kết nối Wi-Fi 13
3.2.4 Cơ chế Access Point và khôi phục cấu hình 15
3.2.5 Đánh giá giải pháp 15
3.3 Task 5: Huấn luyện và triển khai TinyML 15
3.3.1 Huấn luyện mô hình từ dữ liệu thời tiết 15
3.3.2 Tiền xử lý và cấu trúc mạng 16
3.3.3 Quy trình chuyển đổi mô hình từ .h5 sang .tflite và .h 16
3.3.4 Triển khai mô hình trên ESP32 17
3.3.5 Kết quả hiển thị trên giao diện web 17
3.3.6 Kiểm chứng bằng log terminal 18
3.4 Task 6: Gửi dữ liệu lên máy chủ CoreIOT Cloud 19
3.4.1 Dashboard 19
3.4.2 Cơ chế gửi email - Rule chain 20
4 Kết quả thực nghiệm và Đánh giá **23**
4.1 Hình ảnh thực tế mô hình 23
4.1.1 Mô hình phần cứng sau tích hợp 23
4.1.2 Đặc điểm vận hành quan sát được 23
4.2 Đánh giá độ chính xác của TinyML 23
4.2.1 Tiêu chí đánh giá độ chính xác 23
4.2.2 Nhận xét từ kết quả triển khai thực tế 24

4.2.3 Đánh giá tổng quan 24
4.3 Kết quả kiểm thử 24
4.3.1 Các kịch bản kiểm thử chính 24
4.3.2 Nhận xét sau kiểm thử 25
5 Kết luận và Hướng phát triển 26
5.1 Kết luận 26
5.2 Hướng phát triển tương lai 26

Danh sách hình vẽ

1 FSM các trạng thái của LED đơn tương ứng với từng mức nhiệt độ 8
2 FSM các trạng thái của RGB LED tương ứng với từng mức độ ẩm 9
3 FSM các trạng thái của LCD 11
4 Khối hiển thị tức thời trên giao diện chính của Web Server 12
5 Biểu đồ thời gian thực của nhiệt độ và độ ẩm trên giao diện Web Server 13
6 Trang cấu hình Wi-Fi của Web Server 14
7 Danh sách mạng Wi-Fi quét được và cường độ tín hiệu tương ứng 14
8 Kết quả dự đoán thời tiết của mô hình TinyML trên giao diện web 18
9 Log terminal của mô hình TinyML khi chạy trên ESP32-S3 18
10 Giao diện mặc định của Dashboard 19
11 Giao diện Dashboard cho thiết bị ESP32_001 kết nối với Gateway 20
12 Rule chain hiện thực cơ chế gửi email 21
13 Email tổng hợp thông tin dữ liệu thu được từ các ESP32 22

1 Giới thiệu chung

1.1 Mục tiêu dự án

Bài tập này được thực hiện nhằm đạt được các mục tiêu then chốt về cả kỹ năng chuyên môn lẫn tư duy hệ thống trong lĩnh vực kỹ thuật nhúng:

- **Làm chủ hệ điều hành thời gian thực:** Tiếp cận và vận hành thành thạo các cơ chế quản lý đa nhiệm trên nền tảng RTOS, từ đó tối ưu hóa việc phân phối tài nguyên hệ thống và đảm bảo tính đáp ứng tức thời cho các tác vụ ưu tiên.
- **Kỹ thuật lập trình nhúng chuyên sâu:** Nâng cao kỹ năng lập trình hướng đối tượng và quản lý mã nguồn hiệu quả bằng cách áp dụng các cơ chế đồng bộ hóa dữ liệu hiện đại, thay thế hoàn toàn các phương pháp truyền dữ liệu truyền thống thông qua biến toàn cục để tăng tính đóng gói và an toàn cho hệ thống.
- **Ứng dụng trí tuệ nhân tạo tại biên:** Nghiên cứu và hiện thực hóa các thuật toán học máy tối giản để xử lý dữ liệu ngay trên thiết bị nhúng. Mục tiêu là giúp hệ thống có khả năng tự đưa ra quyết định thông minh dựa trên dữ liệu cảm biến mà không cần phụ thuộc vào sức mạnh tính toán của máy chủ trung tâm.
- **Xây dựng giải pháp IoT toàn diện:** Thiết kế một hệ sinh thái khép kín từ tầng thiết bị cuối đến tầng giao diện người dùng và hạ tầng lưu trữ đám mây. Dự án tập trung vào việc đảm bảo sự kết nối liên tục, bảo mật dữ liệu và khả năng tương tác linh hoạt giữa người dùng và phần cứng qua mạng không dây.
- **Tư duy phát triển và mở rộng hệ thống:** Hình thành khả năng phân tích, cải tiến và nâng cấp các mã nguồn sẵn có để tạo ra những giải pháp công nghệ mới có tính ứng dụng cao, đáp ứng các tiêu chuẩn kỹ thuật khắt khe trong thực tế.

1.2 Phạm vi thực hiện

Dự án được giới hạn và thực hiện dựa trên 6 nhiệm vụ cụ thể như sau:

- **Nền tảng phát triển:** Toàn bộ quá trình lập trình và phát triển được thực hiện trên nền tảng vi điều khiển ESP32 sử dụng công cụ PlatformIO.
- **Giao tiếp cảm biến và hiển thị:** Thực hiện thu thập dữ liệu nhiệt độ và độ ẩm để hiển thị lên màn hình LCD thông qua chuẩn giao tiếp I2C. Hệ thống xử lý điều kiện để điều khiển LED đơn chớp tắt và thay đổi màu sắc LED NeoPixel dựa trên các ngưỡng môi trường, đồng thời sử dụng Semaphore và Message Queue để đồng bộ các trạng thái bình thường, cảnh báo hoặc nguy hiểm.

- **Quản lý Web Server:** Thiết kế lại giao diện Web Server hỗ trợ điều khiển ít nhất 1 thiết bị. Hệ thống hoạt động với chế độ Access Point cho lần thiết lập đầu tiên và chuyển sang chế độ Station cho hoạt động bình thường.
- **Triển khai TinyML:** Xây dựng mã nguồn để thu thập dữ liệu và gán nhãn cho các trạng thái bất thường của nhiệt độ hoặc độ ẩm. Nhóm thực hiện huấn luyện và chạy mô hình phân tích trực tiếp trên thiết bị để đánh giá độ chính xác của hệ thống.
- **Tích hợp điện toán đám mây:** Vi điều khiển ESP32 ở chế độ STA sẽ kết nối và đẩy dữ liệu môi trường lên hệ thống CoreIOT thông qua mã token riêng biệt hoặc sẽ kết nối với một Gateway, rồi từ Gateway đấy sẽ đẩy dữ liệu lên CoreIOT.

2 Thiết kế hệ thống

2.1 Kiến trúc hệ thống

Hệ thống được xây dựng xoay quanh vi điều khiển ESP32-S3 gắn sẵn trên board mạch, đóng vai trò là khối xử lý trung tâm. Với cấu hình phần cứng tương đối gọn nhẹ, thiết bị thực hiện đồng thời nhiều nhiệm vụ như đọc dữ liệu từ bộ cảm biến nhiệt độ – độ ẩm, hiển thị thông tin lên LCD, vận hành Web Server để cấu hình mạng, chạy mô hình TinyML để dự đoán trạng thái thời tiết và gửi dữ liệu lên nền tảng cloud.

Về mặt tổ chức chức năng, hệ thống có thể chia thành ba lớp chính. Lớp thứ nhất là lớp cảm nhận, bao gồm bộ cảm biến nhiệt độ và độ ẩm. Lớp thứ hai là lớp xử lý tại biên, do ESP32-S3 đảm nhiệm, bao gồm xử lý dữ liệu, điều khiển LCD, vận hành Web Server và suy luận TinyML. Lớp thứ ba là lớp dịch vụ mạng, trong đó dữ liệu sau xử lý được đồng bộ lên ThingsBoard để trực quan hóa, lưu trữ và kích hoạt các tác vụ tự động như gửi email. Cách tổ chức này giúp hệ thống vừa giữ được khả năng phản hồi nhanh trên thiết bị, vừa có thể mở rộng lên mức giám sát từ xa.

2.2 Sơ đồ khối và luồng dữ liệu

Luồng dữ liệu của hệ thống được thực hiện theo trình tự như sau:

1. Bộ cảm biến thu thập dữ liệu nhiệt độ và độ ẩm của môi trường.
2. Dữ liệu được truyền về ESP32 để xử lý và phân phối cho các tác vụ nội bộ.
3. Thông tin đo được được hiển thị trực tiếp lên LCD và biểu đạt bằng các trạng thái của LED đơn và LED NeoPixel để người dùng quan sát tại chỗ.
4. Đồng thời, nhiệt độ và độ ẩm được chuẩn hóa và đưa vào mô hình TinyML để suy luận trạng thái thời tiết.

5. Kết quả suy luận cùng dữ liệu cảm biến được hiển thị trên Web Server và gửi tiếp lên app.coreiot.io để phục vụ giám sát từ xa.

Điểm đáng chú ý của luồng xử lý này là toàn bộ phần đo đạc, hiển thị và suy luận quan trọng đều được thực hiện ngay trên ESP32. Nhờ đó, hệ thống không cần phụ thuộc hoàn toàn vào máy chủ ngoài để đưa ra kết quả tức thời. Trong khi đó, nền tảng cloud chỉ đảm nhiệm vai trò mở rộng như lưu trữ, trực quan hóa lịch sử và thông báo, giúp hệ thống có được cả tính thời gian thực lẫn khả năng giám sát từ xa.

2.3 Danh sách linh kiện phần cứng

Do mô hình được thiết kế theo hướng đơn giản và dễ triển khai, phần cứng sử dụng không nhiều. Bảng dưới đây liệt kê các thành phần chính có trong hệ thống.

STT	Linh kiện	Số lượng	Vai trò
1	ESP32 gắn sẵn trên board	1	Vi điều khiển trung tâm, xử lý dữ liệu, vận hành Web Server, chạy TinyML và kết nối CoreIOT
2	DHT20	1	Thu thập dữ liệu môi trường để đưa vào hệ thống giám sát và mô hình TinyML
3	LCD	1	Hiển thị trực tiếp các thông số môi trường và trạng thái hệ thống tại thiết bị
4	Dây bus kết nối	1 bộ	Kết nối các khối phần cứng và truyền tín hiệu giữa cảm biến, LCD và board ESP32
5	Dây Type-C	1	Cấp nguồn và nạp chương trình cho board ESP32

Bảng 1: Danh sách linh kiện phần cứng sử dụng trong mô hình

Nhìn chung, cấu hình phần cứng của mô hình khá tối giản nhưng vẫn đủ để hiện thực một hệ thống IoT hoàn chỉnh. Việc chỉ sử dụng một số linh kiện cơ bản giúp hệ thống dễ lắp ráp, dễ kiểm thử và phù hợp với mục tiêu minh họa rõ quá trình tích hợp giữa cảm biến, vi điều khiển, giao diện người dùng và mô hình học máy nhúng.

3 Triển khai các Task

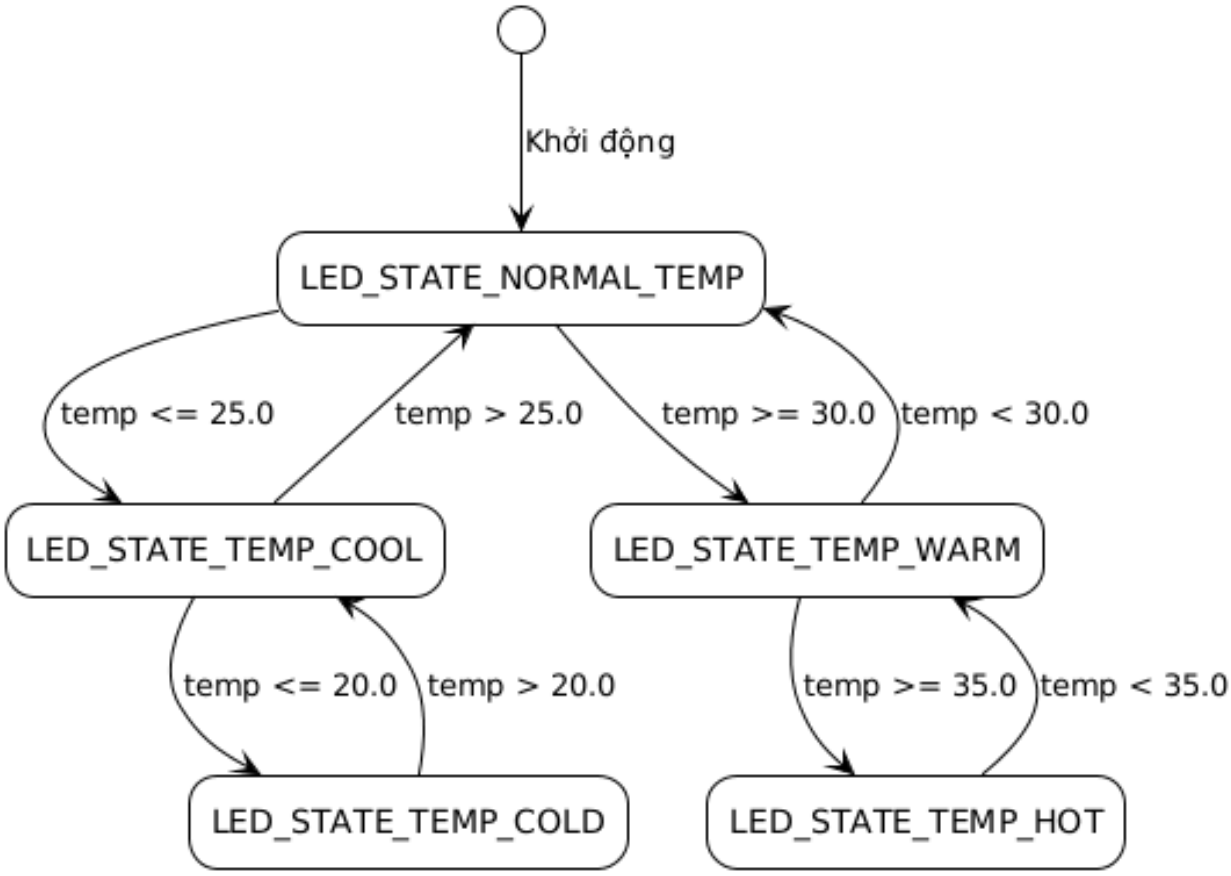
3.1 Task 1,2, 3: Thu thập dữ liệu và trực quan hóa

3.1.1 Tiến trình Thu thập Dữ liệu

Nội dung trọng tâm được triển khai trong phần này là xây dựng cơ chế thu thập thông tin khí hậu chính xác từ cảm biến kỹ thuật số DHT20. Sau quá trình thu thập dữ liệu, tất cả những thông tin về nhiệt độ và độ ẩm sẽ được lưu trữ trong Message Queue. Từ đấy toàn bộ dữ liệu sẽ được hệ thống phân tích và biểu diễn đa dạng thông qua các giao diện phần cứng khác nhau: từ việc hiển thị ký tự trên màn hình LCD cho đến việc sử dụng các trạng thái ánh sáng của LED đơn và LED RGB, qua đó tạo ra một hệ thống trực quan hóa dữ liệu sinh động và dễ dàng quan sát.

3.1.2 Hiển thị trên LED đơn với các trạng thái nhiệt độ khác nhau

Tác vụ quản lý thiết bị hiển thị LED đơn chịu trách nhiệm biểu diễn trạng thái nhiệt độ thông qua các nhịp điệu chớp tắt khác nhau. Hệ thống phân loại điều kiện môi trường dựa trên kiểu liệt kê LEDBlinkyState_t với năm cấp độ riêng biệt. Mỗi cấp độ sẽ tương ứng với một khoảng thời gian được bộ định thời của hệ điều hành quản lý chặt chẽ nhằm đảm bảo tính chính xác về mặt thời gian thực.



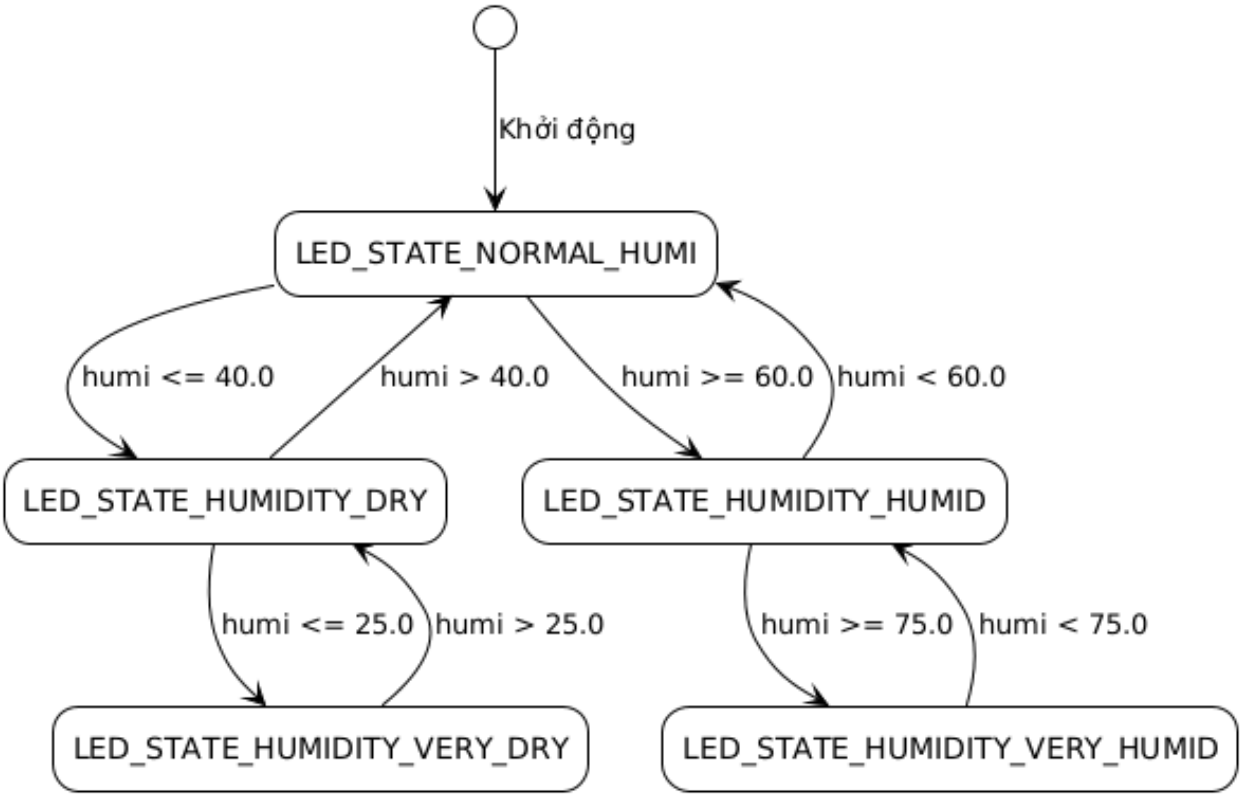
Hình 1: FSM các trạng thái của LED đơn tương ứng với từng mức nhiệt độ

- LED_STATE_TEMP_HOT**:** Trạng thái này xuất hiện khi nhiệt độ đo được vượt ngưỡng 35°C. Để cảnh báo tình trạng quá nhiệt nghiêm trọng, đèn LED sẽ được điều khiển chớp tắt với tần suất cực nhanh là 100 mili giây cho mỗi chu kỳ.
- LED_STATE_TEMP_WARM**:** Khi nhiệt độ nằm trong khoảng từ 30°C đến dưới 35°C, hệ thống xác định đây là vùng cảnh báo mức độ thấp. Nhịp điệu của đèn LED lúc này được duy trì ổn định ở mức 300 mili giây.
- LED_STATE_NORMAL_TEMP**:** Đây là dải nhiệt độ lý tưởng cho vận hành, được xác lập khi giá trị đo được nằm giữa 25°C và 30°C. Đèn LED duy trì nhịp chớp tắt tiêu chuẩn là 1000 mili giây để biểu thị hệ thống đang hoạt động trong điều kiện bình thường.
- LED_STATE_TEMP_COOL**:** Trong trường hợp môi trường trở nên mát mẻ hơn với nhiệt độ từ trên 20°C đến 25°C, hệ thống sẽ chuyển sang trạng thái này. Tần suất hoạt động của đèn sẽ chậm lại đáng kể với chu kỳ là 2000 mili giây.
- LED_STATE_TEMP_COLD**:** Đây là mức nhiệt độ thấp nhất khi giá trị xuống dưới hoặc bằng 20°C. Hệ thống phát tín hiệu bằng cách chớp tắt LED rất chậm với khoảng thời gian lên đến 3000 mili giây giữa các lần chuyển đổi trạng thái.

Ngoài ra, việc điều khiển chân tín hiệu của LED còn chịu ảnh hưởng trực tiếp từ tín hiệu RPC được gửi từ app.coreiot.io. Các tín hiệu đó sẽ làm thay đổi xSemaphoreLedControl. Chỉ khi lấy được thành công Semaphore này, hệ thống cho phép LED hoạt động dựa trên logic thời gian như trên và sau đó hoàn trả quyền điều khiển. Ngược lại nếu không lấy được Semaphore, đèn LED đơn sẽ bị ép về trạng thái tắt.

3.1.3 Hiển thị RGB LED tương ứng với các trạng thái độ ẩm khác nhau

Phân hệ hiển thị đa sắc sử dụng dải RGB LED để cung cấp cái nhìn trực quan về độ ẩm không khí hiện tại và trạng thái mạng. Các trạng thái màu sắc được định nghĩa thông qua kiểu dữ liệu NeoBlinkyState_t và phụ thuộc chặt chẽ vào độ ẩm hiện tại. Trong khi đó, trạng thái bật tắt của đèn sẽ do trạng thái kết nối mạng hiện tại quyết định.



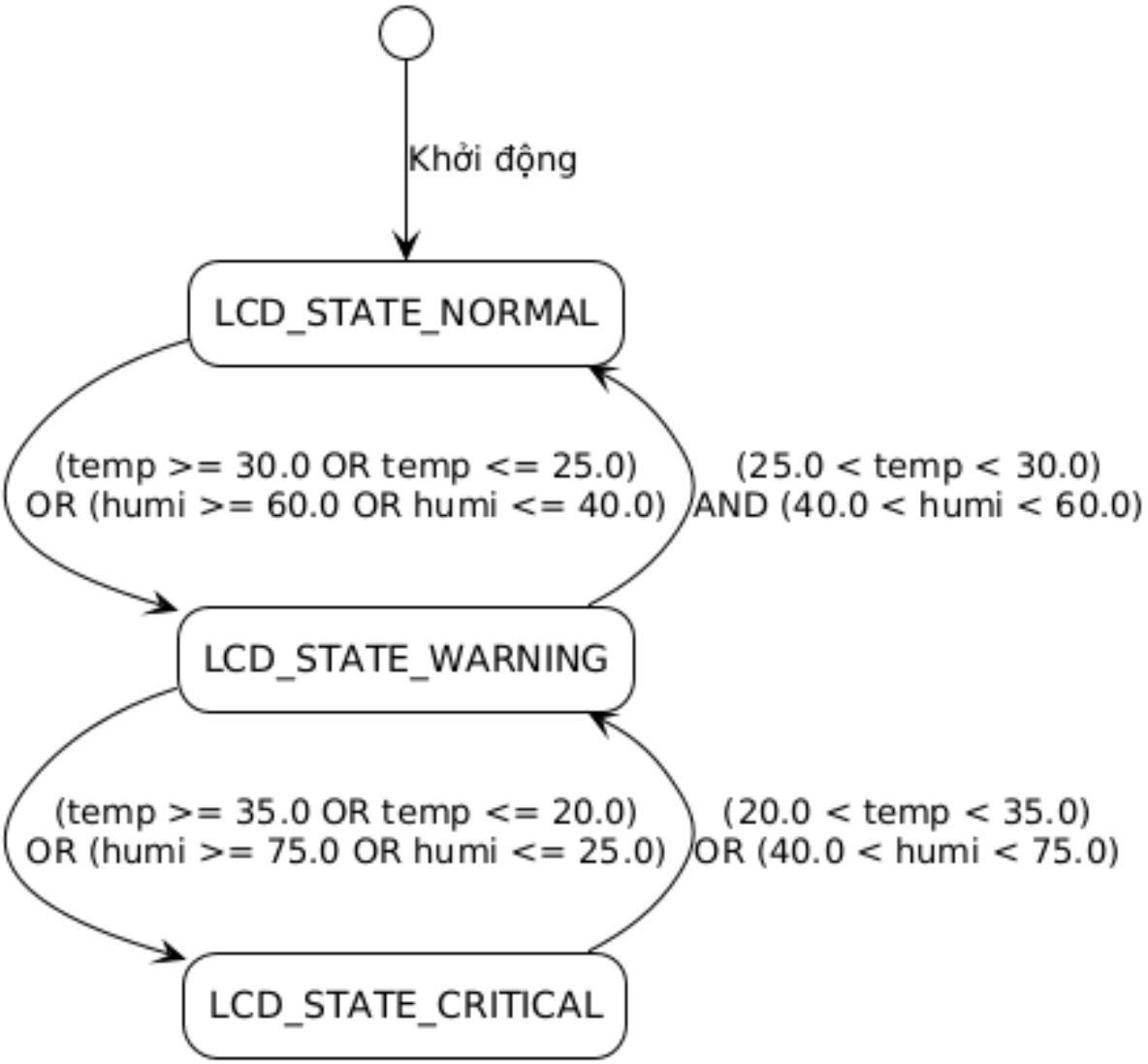
Hình 2: FSM các trạng thái của RGB LED tương ứng với từng mức độ ẩm

- **Quản lý trạng thái kết nối:** Hệ thống ưu tiên kiểm tra cờ trạng thái của Semaphore kết nối mạng xBinarySemaphoreInternet. Nếu mất tín hiệu đường truyền, thiết bị sẽ phản hồi bằng cách cho dải LED chớp tắt liên tục mỗi 500 mili giây. Điểm đặc biệt là trong pha sáng, màu sắc vẫn phản ánh đúng mức độ ẩm hiện tại để người dùng không bị gián đoạn việc giám sát môi trường. Khi kết nối ổn định, dải LED sẽ chuyển sang chế độ sáng liên tục với màu sắc tương ứng.

- **Phân loại theo mức độ ẩm:** Màu sắc của dải LED được nội suy theo năm vùng giá trị cụ thể như sau:
  - LED_STATE_HUMIDITY_VERY_HUMID**:** Khi độ ẩm đạt mức rất cao từ 75% trở lên, đèn chuyển sang màu Tím nhằm cảnh báo độ ẩm không khí các ở mức cao.
  - LED_STATE_HUMIDITY_HUMID**:** Với dải độ ẩm từ 60% đến dưới 75%, hệ thống hiển thị màu Lam để biểu thị môi trường đang ở mức khá ẩm ướt.
  - LED_STATE_NORMAL_HUMI**:** Màu Lục được chọn để biểu thị dải độ ẩm bình thường, dao động từ 40% đến dưới 60%.
  - LED_STATE_HUMIDITY_DRY**:** Khi không khí bắt đầu khô hanh với độ ẩm từ 25% đến dưới 40%, đèn sẽ chuyển sang màu Vàng để nhắc nhở người dùng.
  - LED_STATE_HUMIDITY_VERY_DRY**:** Nếu độ ẩm rơi xuống mức cực thấp dưới 25%, hệ thống lập tức hiển thị màu Đỏ để cảnh báo về điều kiện môi trường quá khô.

3.1.4 Hiển thị LCD

Màn hình tinh thể lỏng đóng vai trò là trung tâm thông báo chi tiết của hệ thống. Dựa trên các thông số thu thập được, máy trạng thái của màn hình được điều khiển thông qua cấu trúc liệt kê LCDState_t với ba chế độ hoạt động có tính phân cấp:



Hình 3: FSM các trạng thái của LCD

- LCD_STATE_NORMAL: Trong điều kiện môi trường ổn định, màn hình hiển thị trực tiếp các con số về nhiệt độ và độ ẩm một cách rõ ràng. Nội dung được định dạng lại để tối ưu hóa không gian hiển thị vốn hạn hẹp của các dòng ký tự, giúp người dùng nắm bắt nhanh các thông số cơ bản.

- LCD_STATE_WARNING: Khi một trong hai chỉ số môi trường bắt đầu rời khỏi vùng lý tưởng, hệ thống sẽ tự động chèn thêm thông điệp cảnh báo SOS ngay phía sau giá trị đo được. Việc này tạo ra một sự thay đổi trực quan trên giao diện hiển thị nhằm thu hút sự chú ý của người quan sát mà không làm mất đi các thông tin đo lường cần thiết.

- LCD_STATE_CRITICAL: Đây là trạng thái báo động cao nhất khi nhiệt độ hoặc độ ẩm chạm tới các ngưỡng cực đoan. Lúc này, màn hình sẽ chuyển sang chế độ nhấp nháy liên tục toàn bộ nội dung hoặc đèn nền. Sự thay đổi trạng thái thị giác mạnh mẽ này

là phương thức cảnh báo cấp bách nhất, yêu cầu người vận hành phải có biện pháp can thiệp ngay lập tức để bảo vệ thiết bị hoặc điều chỉnh môi trường.

3.2 Task 4: Web Server ở chế độ Access Point

3.2.1 Mục tiêu và chế độ vận hành

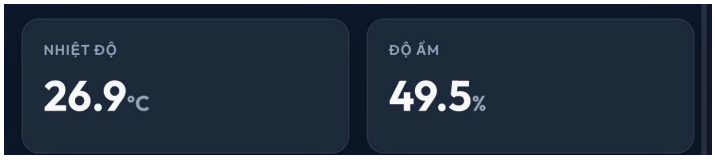
Task 4 tập trung vào việc xây dựng một Web Server chạy trực tiếp trên ESP32-S3 để phục vụ quá trình cấu hình mạng và giám sát thiết bị tại chỗ. Theo thiết kế, thiết bị khởi động ở chế độ *Access Point* trong lần cấu hình đầu tiên để người dùng có thể truy cập giao diện web mà không cần phụ thuộc vào hạ tầng mạng sẵn có. Sau khi nhận được thông tin SSID và mật khẩu hợp lệ, hệ thống sẽ chuyển sang chế độ *Station* để tham gia vào mạng Wi-Fi nội bộ và tiếp tục duy trì giao diện điều khiển từ xa.

Kiến trúc phần mềm của phân hệ này được thể hiện qua tệp khai báo mainserver.h trong thư mục uploads. Các hàm startAP(), connectToWiFi(), setupServer(), mainPage() và settingsPage() cho thấy hệ thống được tổ chức thành hai lớp chức năng rõ ràng: lớp thiết lập kết nối mạng và lớp sinh giao diện HTML để tương tác với người dùng. Toàn bộ phần phục vụ Web Server được đặt trong tác vụ riêng main_server_task(), nhờ đó hoạt động mạng không làm chặn các tiến trình đọc cảm biến hay cập nhật hiển thị.

3.2.2 Giao diện chính của Web Server

Từ các hằng số phần cứng được định nghĩa trong mã nguồn, hệ thống Web Server điều khiển tối thiểu hai đầu ra số, tương ứng với LED1_PIN = 48 và LED2_PIN = 41. Điều này đáp ứng đúng yêu cầu của bài toán là giao diện web phải hỗ trợ điều khiển ít nhất hai thiết bị độc lập.

Trang điều khiển chính được thiết kế theo hướng trực quan và tối giản. Người dùng có thể quan sát ngay các đại lượng môi trường quan trọng như nhiệt độ và độ ẩm mà không cần thao tác qua nhiều lớp menu. Bên cạnh phần hiển thị số liệu, giao diện còn được thiết kế để hỗ trợ điều khiển tối thiểu hai thiết bị tương ứng với hai đầu ra LED1 và LED2. Theo đặc tả bài tập, đây là hai đối tượng điều khiển cơ bản mà Web Server phải quản lý thông qua các nút nhấn hoặc các hành động điều khiển được gắn nhãn rõ ràng. Cách trình bày này phù hợp với các hệ thống giám sát IoT, nơi ưu tiên hàng đầu là tốc độ nhận biết trạng thái hiện tại của thiết bị kết hợp với khả năng phát lệnh điều khiển nhanh.



Hình 4: Khối hiển thị tức thời trên giao diện chính của Web Server

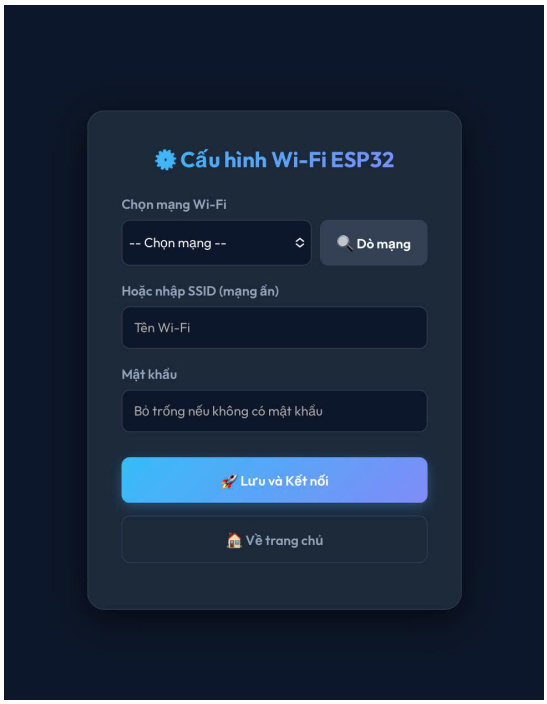
Bên cạnh phần số liệu tức thời, giao diện còn tích hợp biểu đồ thời gian thực để mô tả sự thay đổi của nhiệt độ và độ ẩm theo thời gian. Nhờ đó, người dùng không chỉ biết giá trị hiện tại mà còn theo dõi được xu hướng tăng giảm của dữ liệu cảm biến trong một khoảng thời gian ngắn.

Hình 5: Biểu đồ thời gian thực của nhiệt độ và độ ẩm trên giao diện Web Server

Sự kết hợp giữa phần số liệu tức thời, biểu đồ lịch sử ngắn hạn và các nút điều khiển giúp trang chính thực hiện tốt ba vai trò cùng lúc: giám sát nhanh, quan sát xu hướng và phát lệnh tới thiết bị. Đây là điểm thiết kế quan trọng giúp Web Server không chỉ là giao diện cấu hình mà còn là một bảng điều khiển cục bộ đúng với yêu cầu của Task 4.

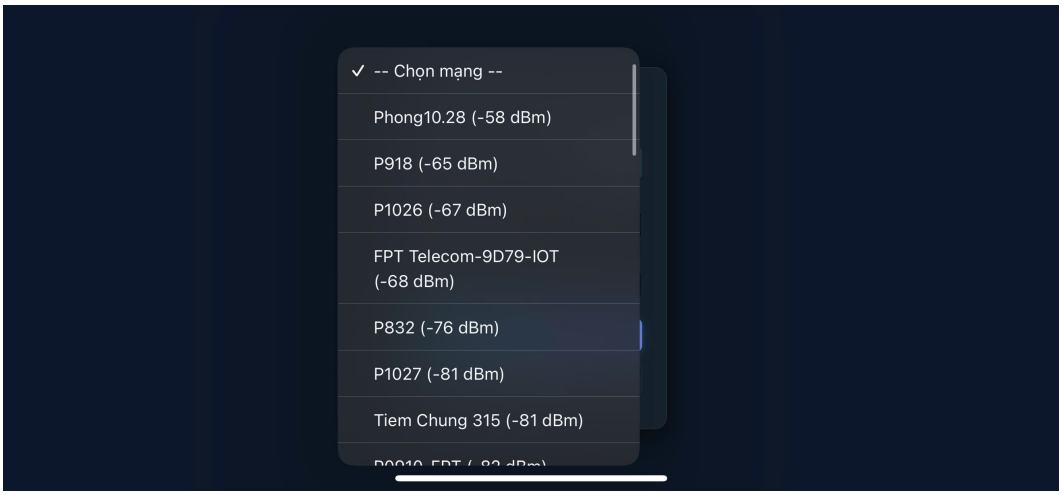
3.2.3 Trang cấu hình và quy trình kết nối Wi-Fi

Hệ thống tách riêng phần cấu hình mạng thành một trang độc lập để người dùng mới có thể thao tác dễ dàng hơn. Tại đây, người dùng có thể chọn mạng Wi-Fi, nhập SSID ẩn nếu cần, điền mật khẩu và lưu cấu hình để thiết bị chuyển từ chế độ Access Point sang Station.



Hình 6: Trang cấu hình Wi-Fi của Web Server

Sau bước nhập thông tin kết nối, hệ thống hỗ trợ quét danh sách các mạng khả dụng cùng cường độ tín hiệu. Chi tiết này rất hữu ích trong môi trường có nhiều mạng Wi-Fi hoạt động đồng thời, vì nó giúp người dùng chọn đúng điểm truy cập và đánh giá sơ bộ chất lượng tín hiệu trước khi lưu cấu hình.



Hình 7: Danh sách mạng Wi-Fi quét được và cường độ tín hiệu tương ứng

Hai hình trên thể hiện rõ quy trình cấu hình của hệ thống: trước tiên là nhập hoặc chọn thông tin mạng, sau đó là đối chiếu danh sách mạng quét được để hoàn tất kết nối. Cách tổ chức này làm giảm sai sót khi cấu hình và nâng cao trải nghiệm người dùng trong giai đoạn cài đặt ban đầu.

3.2.4 Cơ chế Access Point và khôi phục cấu hình

Một điểm đáng chú ý trong thiết kế là sự hiện diện của chân BOOT_PIN = 0. Trong bối cảnh hệ thống nhúng, đây là tín hiệu phù hợp để triển khai cơ chế đưa thiết bị trở lại trạng thái cấu hình ban đầu hoặc cưỡng bức vào chế độ Access Point khi cần bảo trì. Nhờ cơ chế này, hệ thống không bị phụ thuộc tuyệt đối vào mạng Wi-Fi đã lưu trước đó; người dùng vẫn có thể phục hồi thiết bị ngay cả khi thay đổi router, quên mật khẩu hoặc xảy ra lỗi cấu hình.

Luồng vận hành tổng quát của Task 4 có thể mô tả như sau:

1. Thiết bị khởi động và kiểm tra trạng thái cấu hình mạng.
2. Nếu chưa có cấu hình hợp lệ, ESP32 phát Access Point và khởi tạo Web Server cục bộ.
3. Người dùng truy cập giao diện web, nhập thông tin mạng tại *Settings Page*.
4. Hệ thống gọi connectToWiFi() để chuyển sang chế độ Station và tham gia mạng thật.
5. Sau khi kết nối thành công, giao diện điều khiển tiếp tục được duy trì để thao tác với các thiết bị ngoại vi.

3.2.5 Đánh giá giải pháp

Giải pháp Web Server ở chế độ Access Point mang lại nhiều ưu điểm thực tiễn cho hệ thống IoT. Thứ nhất, quá trình cấu hình ban đầu trở nên độc lập với hạ tầng mạng sẵn có, rất phù hợp trong các bài toán triển khai ngoài hiện trường. Thứ hai, việc đóng gói toàn bộ xử lý mạng trong một tác vụ riêng giúp tăng tính mô-đun và giảm nguy cơ ảnh hưởng tới các tác vụ thời gian thực khác. Cuối cùng, cách sắp xếp giao diện thành từng thành phần rõ ràng giúp người dùng dễ làm quen và giúp báo cáo thể hiện đúng ý tưởng thiết kế của Task 4.

3.3 Task 5: Huấn luyện và triển khai TinyML

3.3.1 Huấn luyện mô hình từ dữ liệu thời tiết

Task 5 tập trung vào việc xây dựng một pipeline hoàn chỉnh để huấn luyện mô hình trên máy tính, chuyển đổi mô hình sang định dạng phù hợp cho vi điều khiển và triển khai suy luận trực tiếp trên ESP32-S3. Quy trình này được thể hiện rõ trong tệp TFL_For_MCU.py thuộc thư mục prism-uploads. Điểm mạnh của cách làm này là toàn bộ chuỗi xử lý được tổ chức nhất quán, từ dữ liệu gốc đến mô hình cuối cùng dùng trong firmware.

Bộ dữ liệu đầu vào là tệp weather_classification_data.csv. Trong kịch bản huấn luyện, chỉ ba cột Temperature, Humidity và Weather Type được giữ lại để phục vụ bài toán phân loại. Đây là một lựa chọn phù hợp với định hướng TinyML, bởi mô hình càng ít đặc trưng đầu vào thì càng nhẹ và càng dễ suy luận nhanh trên phần cứng hạn chế tài nguyên.

Trước khi huấn luyện, dữ liệu được làm sạch bằng phương pháp *Interquartile Range* (IQR) nhằm loại bỏ ngoại lai ở hai cột nhiệt độ và độ ẩm. Từ 13.200 mẫu ban đầu, sau bước lọc còn lại 13.108 mẫu hợp lệ. Các nhãn dự đoán được mã hóa thành bốn lớp Cloudy, Rainy, Snowy và Sunny, với phân bố khá cân bằng giữa các lớp. Việc cân bằng dữ liệu như vậy giúp giảm nguy cơ mô hình thiên lệch về một nhóm thời tiết cụ thể.

3.3.2 Tiền xử lý và cấu trúc mạng

Sau bước làm sạch, dữ liệu được chia theo tỉ lệ 80% cho huấn luyện và 20% cho kiểm thử, tương ứng 10.486 mẫu train và 2.622 mẫu test. Hai đầu vào nhiệt độ và độ ẩm được chuẩn hóa bằng StandardScaler theo công thức:

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó, các hệ số thu được từ tập huấn luyện là:

- **Nhiệt độ:** $\mu_T = 18.6480$, $\sigma_T = 16.6074$.
- **Độ ẩm:** $\mu_H = 68.5105$, $\sigma_H = 20.2033$.

Các hệ số này có vai trò rất quan trọng vì chúng phải được giữ nguyên khi suy luận trên ESP32. Nếu dữ liệu đưa vào vi điều khiển không được chuẩn hóa theo đúng cùng một cách như lúc huấn luyện, đầu ra của mô hình sẽ không còn đáng tin cậy.

Mô hình được xây dựng dưới dạng mạng MLP gọn nhẹ, gồm 2 đầu vào, hai lớp ẩn 16 và 8 neuron dùng hàm kích hoạt ReLU, và một lớp đầu ra 4 neuron dùng *softmax*. Trong mã nguồn, mô hình được huấn luyện với bộ tối ưu adam, batch size 32 và 50 epochs. Cấu trúc này đủ đơn giản để chạy trên thiết bị nhúng, nhưng vẫn đủ linh hoạt để học mối quan hệ giữa nhiệt độ, độ ẩm và trạng thái thời tiết.

3.3.3 Quy trình chuyển đổi mô hình từ .h5 sang .tflite và .h

Sau khi huấn luyện xong, mô hình Keras trước hết được lưu lại dưới dạng TinyML.h5. Đây là định dạng thuận tiện để lưu trữ đầy đủ kiến trúc mạng và trọng số, phù hợp cho giai đoạn phát triển trên máy tính. Tuy nhiên, tệp .h5 chưa thể dùng trực tiếp trên vi điều khiển.

Bước tiếp theo là dùng TFLiteConverter để chuyển mô hình sang định dạng TensorFlow Lite và tạo ra tệp TinyML.tflite. Đây là phiên bản rút gọn, phù hợp hơn cho môi trường nhúng nhờ kích thước nhỏ và cơ chế suy luận tối ưu cho thiết bị biên. Trong dự án hiện tại, tệp này có kích thước khoảng 3 KB, nhỏ hơn đáng kể so với TinyML.h5 ban đầu. Sau đó, kịch bản chuyển đổi tiếp tục đọc nội dung nhị phân của TinyML.tflite, đổi từng byte sang dạng thập lục phân và sinh ra tệp header để nhúng vào firmware. Trong phiên bản tích hợp hiện tại, phần dữ liệu mô hình được sử dụng thông qua tệp toanvinh_model.h, trong đó mô hình được lưu dưới

dạng mảng hằng số C để chương trình ESP32 có thể nạp trực tiếp khi khởi tạo TensorFlow Lite Micro.

Điểm đáng chú ý là phần header của mô hình cũng ghi rõ số lớp và tên các lớp dự đoán, giúp việc tích hợp với firmware dễ hiểu hơn. Vì vậy, Task 5 không chỉ dừng ở bước huấn luyện mô hình, mà còn hiện thực hóa đầy đủ quy trình đóng gói mô hình từ môi trường Python sang môi trường C/C++ nhúng.

3.3.4 Triển khai mô hình trên ESP32

Phía firmware, tệp tinyml.h trong thư mục uploads cho thấy mô hình được sử dụng thông qua các thành phần TensorFlowLite_ESP32, MicroInterpreter và AllOpsResolver. Đồng thời, file này cũng bao gồm toanvinh_model.h, cho thấy đây là tệp mô hình thực tế đang được liên kết vào chương trình. Phân hệ TinyML được đóng gói thành hai phần chính: setupTinyML() để khởi tạo mô hình và tiny_ml_task() để thực hiện suy luận trong lúc hệ thống đang vận hành.

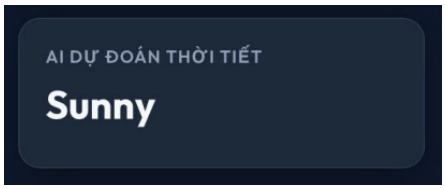
Chu trình suy luận trên ESP32 có thể tóm tắt như sau:

1. Đọc nhiệt độ và độ ẩm từ cảm biến.
2. Chuẩn hóa hai giá trị này bằng đúng các hệ số đã sinh ra từ StandardScaler ở giai đoạn huấn luyện.
3. Ghi dữ liệu đã chuẩn hóa vào tensor đầu vào của mô hình.
4. Thực thi suy luận bằng TensorFlow Lite Micro.
5. Đọc vector xác suất đầu ra và chọn lớp có xác suất lớn nhất làm kết quả dự đoán cuối cùng.

Ưu điểm lớn nhất của hướng triển khai này là hệ thống không cần gửi dữ liệu thô lên máy chủ để phân loại. Toàn bộ quá trình nhận biết trạng thái thời tiết diễn ra ngay trên thiết bị, giúp giảm độ trễ, giảm phụ thuộc mạng và thể hiện rõ tính chất AI at the edge của đề tài.

3.3.5 Kết quả hiển thị trên giao diện web

Sau khi mô hình được nhúng vào ESP32, kết quả suy luận trước hết được biểu diễn trên giao diện web dưới dạng nhãn thời tiết cuối cùng. Cách hiển thị này phù hợp với người dùng vận hành, vì họ chỉ cần nắm được trạng thái dự đoán mà không cần quan tâm đến chi tiết nội bộ của mô hình.

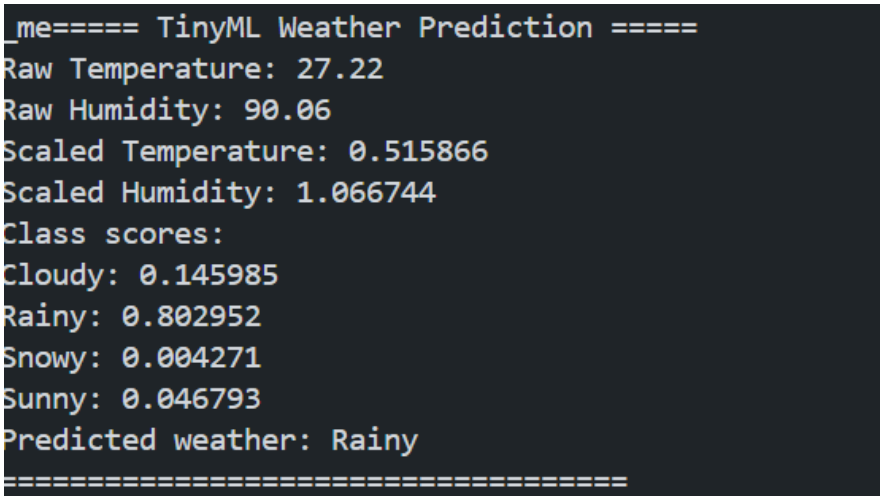


Hình 8: Kết quả dự đoán thời tiết của mô hình TinyML trên giao diện web

Việc đưa trực tiếp nhãn dự đoán lên giao diện web giúp hệ thống tiến thêm một bước so với một mô hình đo lường thông thường: không chỉ hiển thị dữ liệu thô, mà còn cung cấp thông tin đã được xử lý để hỗ trợ người dùng ra quyết định nhanh hơn.

3.3.6 Kiểm chứng bằng log terminal

Bên cạnh giao diện web, hệ thống còn xuất log chi tiết ra terminal để phục vụ quá trình kiểm thử kỹ thuật. Nội dung log bao gồm giá trị đầu vào từ cảm biến, dữ liệu sau chuẩn hóa, xác suất của từng lớp và nhãn dự đoán cuối cùng. Đây là kênh rất quan trọng để xác nhận rằng mô hình sau khi chuyển đổi từ .h5 sang .tflite rồi sang .h vẫn giữ được logic suy luận nhất quán.



```
_me===== TinyML Weather Prediction =====
Raw Temperature: 27.22
Raw Humidity: 90.06
Scaled Temperature: 0.515866
Scaled Humidity: 1.066744
Class scores:
Cloudy: 0.145985
Rainy: 0.802952
Snowy: 0.004271
Sunny: 0.046793
Predicted weather: Rainy
=====================================
```

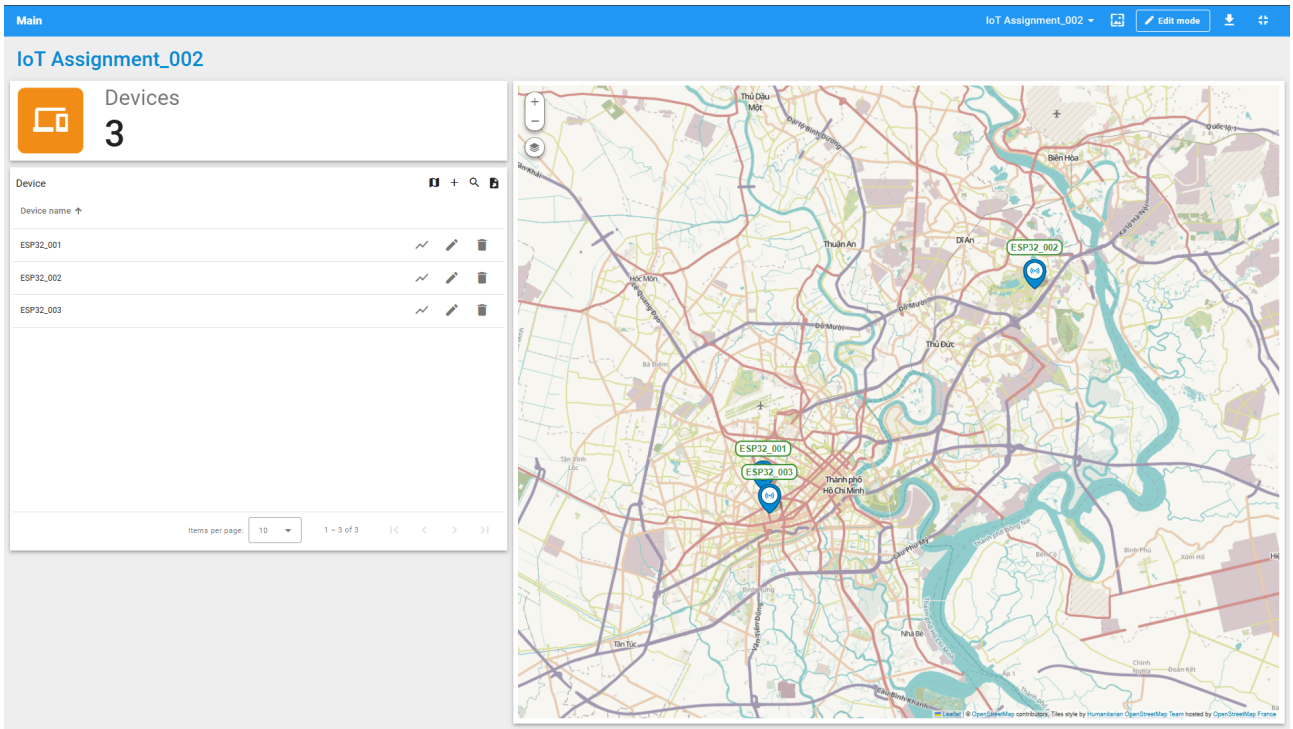
Hình 9: Log terminal của mô hình TinyML khi chạy trên ESP32-S3

Thông qua log terminal, người phát triển có thể kiểm tra được toàn bộ chuỗi xử lý nội bộ của mô hình thay vì chỉ nhìn vào kết quả cuối. Đây là bằng chứng thực nghiệm quan trọng cho thấy mô hình không chỉ chạy được trong môi trường Python mà còn hoạt động ổn định sau khi được chuyển đổi và nhúng vào hệ thống nhúng.

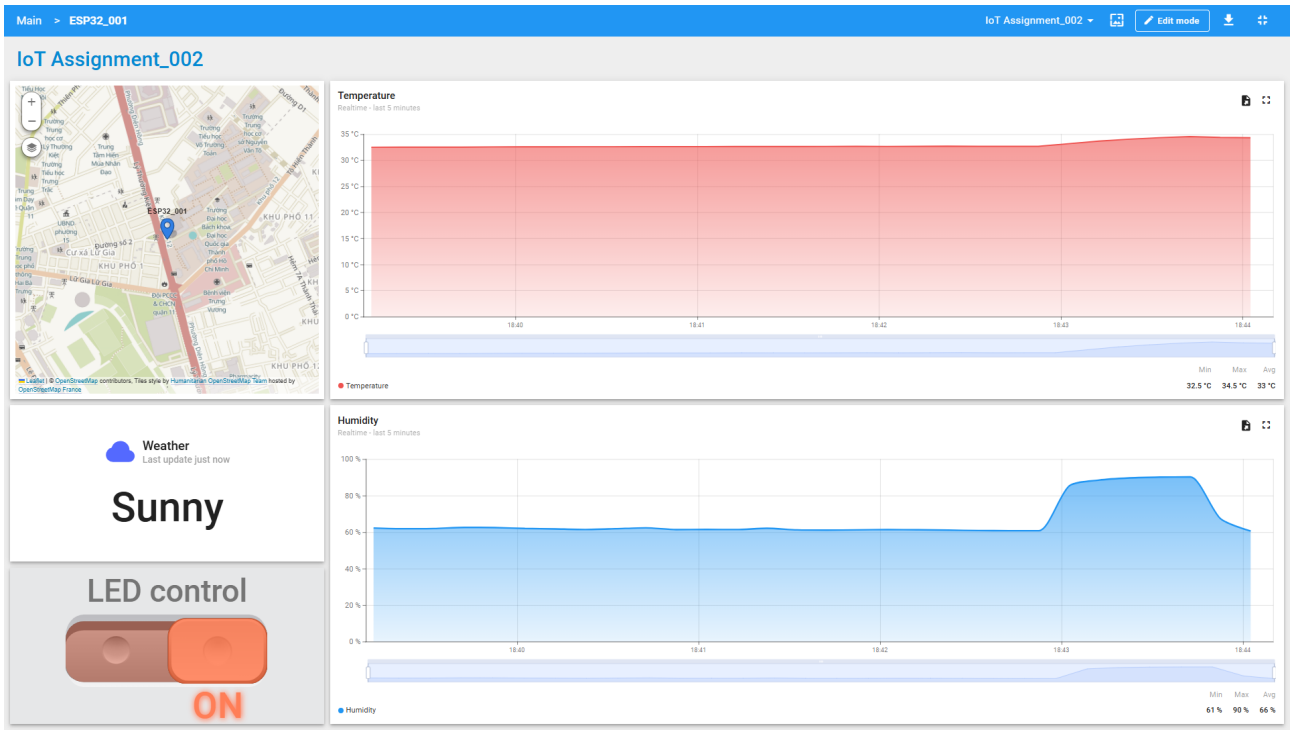
3.4 Task 6: Gửi dữ liệu lên máy chủ CoreIOT Cloud

3.4.1 Dashboard

Dashboard được thiết kế trên nền tảng Collect nhằm mục đích trực quan hóa dữ liệu thời gian thực từ các thiết bị ESP32, giúp người dùng dễ dàng theo dõi trạng thái môi trường và đưa ra các quyết định điều khiển kịp thời.



Hình 10: Giao diện mặc định của Dashboard



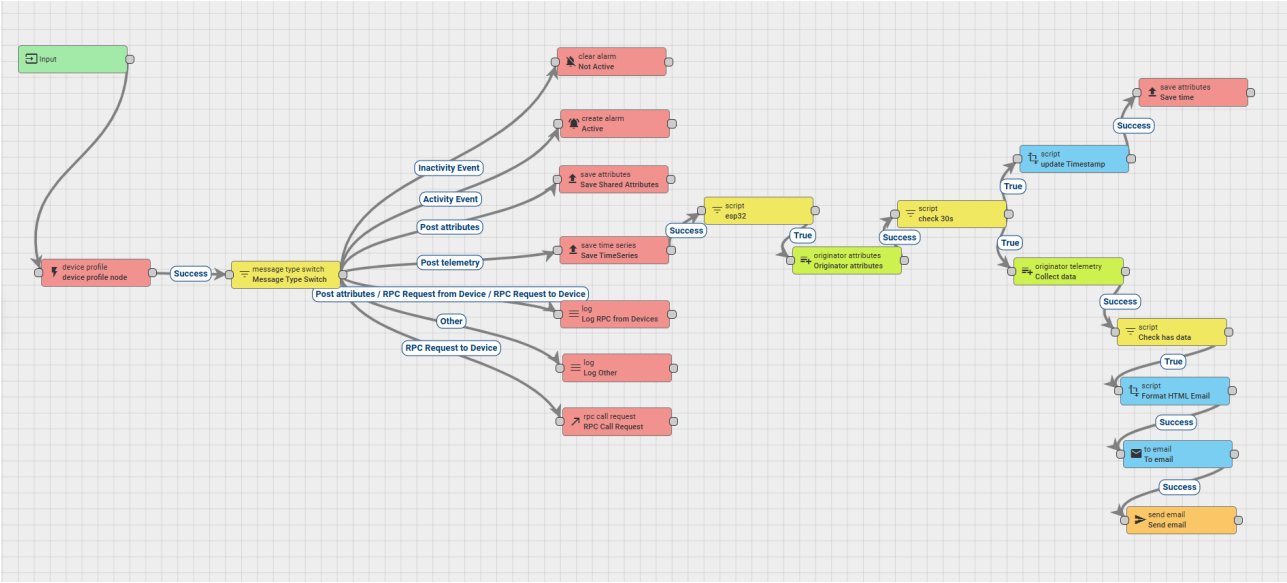
Hình 11: Giao diện Dashboard cho thiết bị ESP32_001 kết nối với Gateway

Dưới đây là các đặc tính kỹ thuật và cấu trúc thành phần của Dashboard "IoT Assignment_002"với cấu hình chung gồm:

- *Giao diện mặc định:* Dashboard được đặt tên là "IoT Assignment_002"với giao diện mặc định thể hiện các device kết nối với Gateway và vị trí của chúng trên bản đồ, nhằm cho mọi người biết số lượng device cũng như phân bố của chúng.
- *Giao diện dữ liệu được gửi từ các device:* Giao diện thứ hai được thể hiện cách thông tin thu thập được từ device như vị trí, nhiệt độ, độ ẩm, kết quả của TinyML. Cùng với đó là sự xuất hiện của một Button gửi tín hiệu RPC về cho ESP32, theo như thiết kế đây là nơi sẽ quyết định LED đơn có bật hay tắt hay không.

3.4.2 Cơ chế gửi email - Rule chain

Hệ thống điều khiển và giám sát sử dụng công cụ Rule Chain của CoreIOT để thiết lập một quy trình xử lý dữ liệu tự động. Quy trình này không chỉ dừng lại ở việc lưu trữ dữ liệu mà còn thực hiện các phép tính thống kê và gửi báo cáo định kỳ qua Email cho người quản lý.



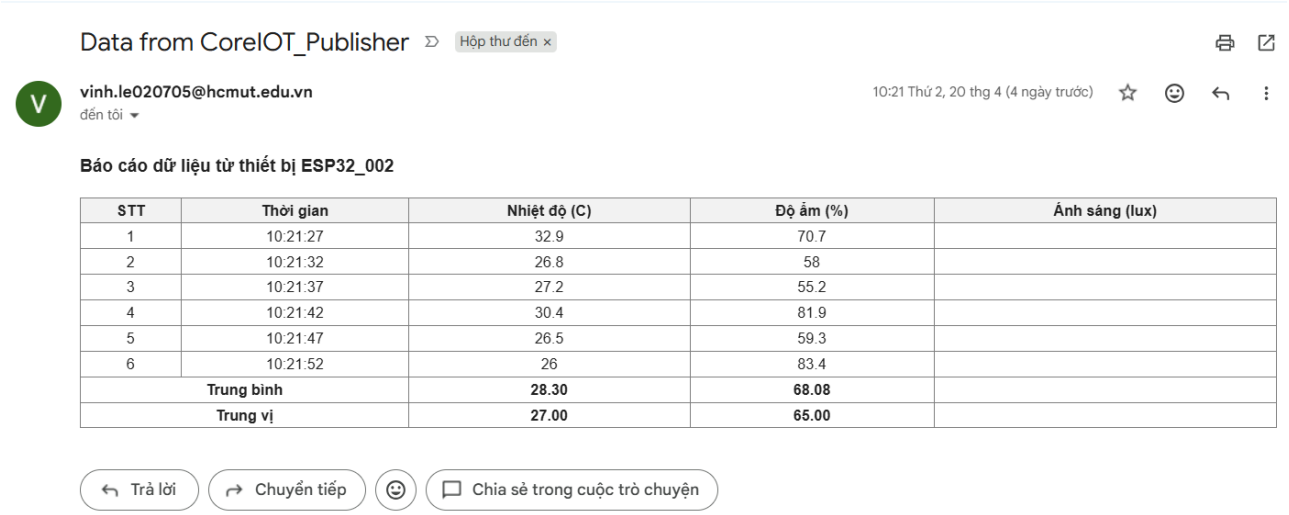
Hình 12: Rule chain hiện thực cơ chế gửi email

Dưới đây là phân tích chi tiết các khối chức năng (Nodes) trong Rule Chain "Gateway":

- **Khối phân loại và lưu trữ cơ sở:**
  - *Message Type Switch:* Đây là node điều hướng trung tâm, có nhiệm vụ phân loại các bản tin đến. Các bản tin thuộc loại "Post telemetry"(dữ liệu cảm biến) sẽ được dẫn hướng tới các bước xử lý báo cáo.
  - *Save TimeSeries:* Thực hiện lưu trữ dữ liệu nhiệt độ, độ ẩm vào cơ sở dữ liệu để hiển thị lên Dashboard. Chỉ khi dữ liệu được lưu thành công, luồng gửi email mới được kích hoạt qua nhánh "Success".
- **Khối lọc thiết bị và kiểm soát tần suất (Rate Limiting):**
  - *esp32 (Filter Node):* Sử dụng mã JavaScript để kiểm tra tên thiết bị có phải ESP32 hay không. Điều này đảm bảo hệ thống chỉ xử lý báo cáo cho các node cảm biến ESP32 mà không bị lẫn với các thiết bị khác.
  - *Originator attributes & check 30s:* Hai khối này phối hợp để ngăn chặn việc gửi email quá dồn dập. Hệ thống truy xuất thuộc tính last_email_time từ server và so sánh với thời gian hiện tại. Theo logic code quy định khoảng cách giữa 2 lần gửi phải $\geq$ 30,000ms.
- **Khối truy vấn và xử lý dữ liệu thống kê:**
  - *Collect data:* Khi thỏa mãn điều kiện về thời gian, node này sẽ truy vấn ngược lại cơ sở dữ liệu để lấy toàn bộ các điểm dữ liệu temperature và humidity trong vòng 30 giây gần nhất.

- *Check has data:* Kiểm tra mảng dữ liệu trả về. Nếu mảng rỗng (không có dữ liệu trong 30 giây qua), quy trình sẽ dừng lại để tránh gửi email trống.
- *Format HTML Email:* Đây là khối xử lý phức tạp nhất bằng JavaScript. Nó thực hiện:
  * Chuyển đổi Timestamp sang định dạng giờ Việt Nam (HH:mm:ss).
  * Tính toán giá trị **Trung bình (Average)** bằng cách lấy tổng chia cho số lượng mẫu.
  * Tính toán giá trị **Trung vị (Median)** bằng cách sắp xếp mảng và lấy giá trị ở giữa.
  * Xây dựng cấu trúc bảng HTML (<table>) để trình bày dữ liệu chuyên nghiệp.

- **Khối thực thi gửi Email:**
  - *To email:* Thiết lập các thông số Header cho email như người gửi, người nhận và tiêu đề thư.
  - *Send email:* Thực hiện việc gửi thư đi thực tế qua internet.


Data from CorelOT_Publisher Hộp thư đến ×

vinh.le020705@hcmut.edu.vn
đến tôi

10:21 Thứ 2, 20 thg 4 (4 ngày trước)

Báo cáo dữ liệu từ thiết bị ESP32_002

STT	Thời gian	Nhiệt độ (C)	Độ ẩm (%)	Ánh sáng (lux)
1	10:21:27	32.9	70.7	
2	10:21:32	26.8	58	
3	10:21:37	27.2	55.2	
4	10:21:42	30.4	81.9	
5	10:21:47	26.5	59.3	
6	10:21:52	26	83.4	
Trung bình		**28.30**	**68.08**	
Trung vị		**27.00**	**65.00**	

Trả lời Chuyển tiếp Chia sẻ trong cuộc trò chuyện

Hình 13: Email tổng hợp thông tin dữ liệu thu được từ các ESP32

Đánh giá tổng quan: Rule Chain này được thiết kế theo mô hình hướng sự kiện (Event-driven). Việc tách biệt luồng cập nhật mốc thời gian và luồng xử lý HTML giúp hệ thống hoạt động ổn định, đảm bảo tính toàn vẹn của dữ liệu báo cáo ngay cả khi lưu lượng dữ liệu từ các node ESP32 gửi về lớn.

4 Kết quả thực nghiệm và Đánh giá

4.1 Hình ảnh thực tế mô hình

4.1.1 Mô hình phần cứng sau tích hợp

Sau khi hoàn thiện toàn bộ các task, hệ thống được tích hợp thành một mô hình IoT hoàn chỉnh xoay quanh vi điều khiển ESP32. Thiết bị trung tâm thực hiện đồng thời nhiều vai trò: thu thập dữ liệu từ cảm biến nhiệt độ và độ ẩm, điều khiển các phần tử hiển thị cục bộ, vận hành Web Server để cấu hình mạng và giám sát tại chỗ, chạy mô hình TinyML để phân loại trạng thái thời tiết, đồng thời gửi dữ liệu lên nền tảng ThingsBoard để phục vụ giám sát từ xa.

Về phần cứng, mô hình bao gồm cảm biến môi trường DHT, màn hình LCD giao tiếp I2C, LED đơn, dải LED RGB và khối kết nối Wi-Fi tích hợp sẵn trên ESP32. Cách bố trí này cho phép người dùng quan sát trạng thái hệ thống dưới nhiều lớp giao diện khác nhau: trực tiếp trên thiết bị, trên giao diện web nội bộ và trên Dashboard đám mây. Nhờ đó, hệ thống vừa đảm bảo tính trực quan khi vận hành tại chỗ, vừa đáp ứng được yêu cầu giám sát từ xa trong môi trường IoT thực tế.

4.1.2 Đặc điểm vận hành quan sát được

Trong quá trình chạy thử, mô hình cho thấy khả năng hoạt động ổn định giữa các tác vụ song song. Dữ liệu nhiệt độ và độ ẩm được cập nhật liên tục; LCD và hệ thống LED phản hồi đúng với các ngưỡng đã cấu hình; Web Server cho phép truy cập nhanh để theo dõi trạng thái và cấu hình mạng; còn nền tảng cloud tiếp nhận dữ liệu gần thời gian thực để hiển thị biểu đồ và gửi email cảnh báo. Điều này cho thấy kiến trúc hệ thống không chỉ đúng về mặt chức năng đơn lẻ mà còn đảm bảo được tính đồng bộ sau khi tích hợp.

4.2 Đánh giá độ chính xác của TinyML

4.2.1 Tiêu chí đánh giá độ chính xác

Độ chính xác của mô hình TinyML được đánh giá theo hai hướng bổ trợ cho nhau. Thứ nhất là đánh giá ngoại tuyến trong quá trình huấn luyện, sử dụng các chỉ số *Accuracy* và *Confusion Matrix* trên tập kiểm thử để xem mô hình phân biệt bốn lớp Cloudy, Rainy, Snowy và Sunny ở mức độ nào. Đây là bước đánh giá định lượng trong môi trường Python. Thứ hai là đánh giá trực tuyến trên thiết bị ESP32 thông qua kết quả suy luận hiển thị trên giao diện web và log chi tiết xuất ra terminal, nhằm kiểm tra xem mô hình sau khi chuyển sang TensorFlow Lite Micro có còn cho ra hành vi nhất quán hay không.

Cách tiếp cận này có ý nghĩa quan trọng vì một mô hình đạt kết quả tốt trên máy tính chỉ thực sự có giá trị khi vẫn giữ được tính nhất quán sau khi chuyển sang TensorFlow Lite và triển khai trên vi điều khiển. Do đó, ngoài việc quan tâm tới nhãn dự đoán cuối cùng, nhóm

còn kiểm tra cả chuỗi xử lý trung gian gồm dữ liệu thô từ cảm biến, giá trị sau chuẩn hóa và phân bố xác suất ở đầu ra của mô hình. Nói cách khác, phần đánh giá trong báo cáo không chỉ dừng ở câu hỏi "mô hình có đúng không", mà còn kiểm tra "mô hình có còn đúng sau khi được nhúng lên phần cứng hay không".

4.2.2 Nhận xét từ kết quả triển khai thực tế

Qua các ảnh minh họa ở phần Task 5, có thể thấy mô hình TinyML đã vận hành đúng theo thiết kế. Trên giao diện web, hệ thống có khả năng hiển thị trực tiếp nhãn dự đoán thời tiết để người dùng quan sát một cách trực quan. Song song với đó, terminal cung cấp đầy đủ các thông tin kiểm chứng như nhiệt độ ban đầu, độ ẩm ban đầu, giá trị đã chuẩn hóa và xác suất của từng lớp. Cơ chế này giúp xác nhận rằng mô hình trên ESP32 không chỉ đưa ra một kết quả cuối cùng, mà còn thực hiện đầy đủ pipeline suy luận như ở môi trường huấn luyện.

Với đặc trưng đầu vào chỉ gồm hai biến là nhiệt độ và độ ẩm, mô hình vẫn cho thấy khả năng phân loại tốt trong các trường hợp đặc trưng rõ rệt. Chẳng hạn, những mẫu có độ ẩm cao thường nghiêng về lớp Rainy, trong khi các mẫu có nhiệt độ cao và độ ẩm thấp hơn thường có xu hướng thuộc lớp Sunny. Tuy nhiên, các lớp có vùng giá trị chồng lấn nhau như Cloudy và Rainy vẫn có thể tạo ra sai số nhất định. Đây là giới hạn tự nhiên của mô hình gọn nhẹ, đồng thời cũng là đánh đổi hợp lý để đổi lấy tốc độ suy luận nhanh và khả năng chạy trực tiếp trên thiết bị nhúng.

4.2.3 Đánh giá tổng quan

Nhìn chung, mô hình TinyML đáp ứng tốt mục tiêu của đề tài là đưa khả năng nhận biết trạng thái môi trường xuống ngay tại biên mạng. Điểm mạnh lớn nhất là tính gọn nhẹ, tốc độ phản hồi nhanh và khả năng tích hợp chặt chẽ với hệ thống giám sát hiện có. Dù chưa hướng đến một mô hình phân loại phức tạp với số lượng đặc trưng lớn, giải pháp hiện tại đã đủ để minh chứng tính khả thi của việc kết hợp AI với hệ thống IoT trên nền tảng ESP32.

4.3 Kết quả kiểm thử

4.3.1 Các kịch bản kiểm thử chính

Để đánh giá toàn diện hệ thống, nhóm tiến hành kiểm thử theo các nhóm chức năng cốt lõi gồm cấu hình mạng, hiển thị cục bộ, suy luận TinyML và kết nối cloud. Kết quả tổng hợp được trình bày trong Bảng 2.

Bảng 2: Tổng hợp kết quả kiểm thử hệ thống

Hạng mục	**Mô tả kiểm thử**	**Kết quả**
Khởi động AP	Thiết bị phát Access Point và cho phép truy cập trang cấu hình trong lần sử dụng đầu tiên hoặc khi cần cấu hình lại.	Đạt
Quét và lưu Wi-Fi	Giao diện web liệt kê các mạng khả dụng, cho phép nhập SSID ẩn, lưu mật khẩu và chuyển sang chế độ Station.	Đạt
Đọc cảm biến	Dữ liệu nhiệt độ và độ ẩm được cập nhật liên tục từ cảm biến và phản ánh lên các giao diện hiển thị.	Đạt
Hiển thị cục bộ	LCD, LED đơn và LED RGB thay đổi đúng theo trạng thái môi trường và ngưỡng cảnh báo.	Đạt
Suy luận TinyML	Kết quả phân loại thời tiết được tạo ra trên ESP32 và hiển thị đồng thời trên web và terminal.	Đạt
Gửi dữ liệu cloud	Telemetry được đẩy lên ThingsBoard và hiển thị lại trên Dashboard gần thời gian thực.	Đạt
Rule Chain gửi email	Hệ thống tạo báo cáo và gửi email định kỳ theo logic đã cấu hình trong Rule Chain.	Đạt

4.3.2 Nhận xét sau kiểm thử

Kết quả kiểm thử cho thấy các khối chức năng chính đều hoạt động đúng với mục tiêu thiết kế. Đặc biệt, việc hệ thống đồng thời duy trì được hiển thị cục bộ, giao diện web, suy luận TinyML và kết nối cloud chứng minh rằng kiến trúc đa tác vụ trên ESP32 đã được tổ chức hợp lý. Trong các tình huống mất kết nối mạng hoặc cần đổi cấu hình Wi-Fi, cơ chế Access Point vẫn cho phép người dùng đưa thiết bị trở về trạng thái cấu hình mà không cần can thiệp phần mềm phức tạp.

Từ góc độ triển khai thực tế, hệ thống đã đạt được mức hoàn thiện tốt cho một bài toán IoT tích hợp. Các bài kiểm thử không chỉ xác nhận từng module hoạt động đúng, mà còn cho thấy toàn bộ chuỗi xử lý từ cảm biến đến người dùng cuối được khép kín và ổn định.

5 Kết luận và Hướng phát triển

5.1 Kết luận

Sau quá trình thực hiện dự án, nhóm đã xây dựng thành công một hệ thống IoT hoàn chỉnh trên nền tảng vi điều khiển ESP32. Kết quả đạt được cho thấy sự kết hợp hiệu quả giữa các kỹ thuật lập trình nhúng hiện đại và khả năng kết nối điện toán đám mây. Các thành tựu chính của dự án bao gồm:

- **Quản lý tác vụ thời gian thực:** Hệ thống vận hành ổn định nhờ việc áp dụng cơ chế Semaphore để điều phối các nhiệm vụ từ đọc dữ liệu cảm biến đến điều khiển thiết bị ngoại vi. Cách tiếp cận này giúp tối ưu hóa hiệu suất xử lý và loại bỏ triệt để việc sử dụng biến toàn cục trong toàn bộ chương trình.
- **Linh hoạt trong điều khiển và cấu hình:** Web Server được thiết kế cho phép hệ thống tự động chuyển đổi giữa các chế độ mạng khác nhau. Người dùng có thể dễ dàng thiết lập thông số hoặc điều khiển thiết bị thông qua giao diện trực quan ngay trên trình duyệt web.
- **Trí tuệ nhân tạo tại biên:** Nhóm đã hiện thực hóa việc phân loại trạng thái môi trường ngay trên phần cứng bằng mô hình TinyML. Điều này chứng minh khả năng xử lý thông minh của thiết bị nhúng mà không cần phụ thuộc hoàn toàn vào máy chủ bên ngoài.
- **Tích hợp hệ sinh thái đám mây:** Dữ liệu môi trường được truyền tải liên tục và bảo mật lên máy chủ CoreIOT. Các tính năng mở rộng như cập nhật phần mềm từ xa và thông báo khẩn cấp qua email đã được triển khai đầy đủ theo yêu cầu kỹ thuật.

Dự án đã giúp các thành viên nắm vững quy trình phát triển một sản phẩm IoT thực tế, từ khâu thiết kế phần cứng đến lập trình phần mềm đa nhiệm.

5.2 Hướng phát triển tương lai

Dựa trên nền tảng sẵn có, hệ thống có thể được tiếp tục cải tiến và mở rộng theo các hướng chủ đạo như sau:

- **Tối ưu hóa thuật toán học máy:** Nhóm dự kiến thu thập tập dữ liệu lớn hơn với nhiều biến số môi trường để huấn luyện các mô hình mạng nơ-ron sâu. Việc này sẽ giúp nâng cao độ chính xác khi dự báo các tình huống bất thường và giảm thiểu sai số trong các môi trường khắc nghiệt.
- **Nâng cao hiệu quả sử dụng năng lượng:** Nghiên cứu và áp dụng các chế độ tiết kiệm điện chuyên sâu là hướng đi quan trọng để kéo dài thời gian hoạt động của thiết bị khi sử dụng nguồn pin. Hệ thống sẽ được lập trình để chỉ kích hoạt các module truyền thông khi có sự thay đổi đáng kể về dữ liệu.

- **Mở rộng khả năng ứng dụng thực tế:** Hệ thống có thể tích hợp thêm các loại cảm biến đo nồng độ bụi mịn hoặc độ pH của đất để phục vụ cho các giải pháp giám sát môi trường đô thị và nông nghiệp công nghệ cao.
- **Tăng cường bảo mật hệ thống:** Việc áp dụng các giao thức mã hóa dữ liệu đầu cuối và cơ chế xác thực đa lớp sẽ là ưu tiên hàng đầu. Điều này đảm bảo an toàn thông tin khi hệ thống được triển khai trong các mạng lưới quy mô lớn hoặc các khu vực nhạy cảm về dữ liệu.

Link Github:

https://github.com/vinhle275/IoT-Assignment